

TaskManager Crud

Realizado por Gabriel Blanco

Este proyecto propio mio, durante mi aprendizaje junto a SpringBoot

Este proyecto se compone de una Arquitectura a Capas, con el lenguaje de Programacion de Java junto al Framework Spring

Dependencias Usadas :

* Spring Web → Este plugin nos ayuda a iniciar nuestra aplicación sobre un Host y recibir peticiones HTTP, las cuales con operaciones CRUD podremos hacer ciertas acciones, mas adelante las mencionare.

* JpaRepository → Nos facilita el proceso de cuando nuestra llave primaria sobre nuestro modelo base , recibe la Clase y tipo de dato especificado ahi , para tomarlo de base para manejar nuestros datos , para que luego se comuniquen con nuestra tabla.

* PostgreSQL → Este nos ayuda a simplificar la conexión de todo nuestro proceso con datos procesados según las reglas del negocio para almacenarlo en nuestra Base de datos.

¿Como funciona en este proyecto la arquitectura en Capas?

Iniciamos con nuestro model, importamos nuestro paquete principal ruta modelo, luego nuestras dependencias necesarias, declaramos con Entidad y tambien apuntando a una tabla en especial, nos da la capacidad de usar metodos Getter y Setter, tambien el id y valor generador para la variable general que tendra un autoIncremento con su estrategia.

Las variables que necesite y se vayan a interactuar en el proyecto, y instanciarlas con el constructor y otro vacio para la reutilizacion de la clase sin parametros.

Repository

Esta no es una clase, es una interfaz que maneja el acceso a los datos unicamente , extendiendo el uso de JpaRepository gracias a Spring junto a nuestro molde y tipo de dato en especial que instanciara con los datos que vaya interactuar con la llave primaria, especificamos @Repository

Service

Esta clase es la que va almacenar las reglas del negocio , importando clases como el Model, Repository, y el Service de Spring stereotype y listas para almacenar los datos en una lista.

La declaramos como servicio, instanciamos el repository a traves de una variable para el uso de los metodos que nos da spring y dar las reglas del negocio, en este CRUD, Create, update , update, delete

Para evitar daños o confusiones usamos constructores parametrizados para instanciar el repository.

Para llamar a todas las tareas usaremos de tipo List, para llamar a todas junto a la clase molde que tiene los datos , junto al metodo findAll() , para llamar a todo

Luego para crear una tarea , usamos igualmente la clase Task y recibira de variable de input la misma clase, y la ejecucion retorna el repository con el metodo save y la tarea dentro asi guardar y almacenara persistencia de los datos en la Base de Datos.

La funcion que completa la tarea recibira la variable que se identifica con la llave primaria junto a su tipo de dato, el cual creara una funcion con la clase Model con un nombre y el metodo que nos otorga Jpa, .findById(id) , funciona como un condicional si lo encuentra , con la variable de la clase instanciada al punto de referencia del Model y el metodo .setCompleted de la misma clase, sera de estado true, si no, retornara una excepcion de ejecucion indicando que la tarea no fue encontrada

Para eliminar usamos la variable mas el tipo de dato de la llave primaria y el metodo .deleteById()

Para buscar por un id en especifico usamos el metodo usado para completar la tarea pero unicamente ese sin instanciar la clase.

Para actualizar una tarea necesitaremos dos atributos de entrada, nuestra llave primaria y la clase Task ,

Instanciaremos la clase para crear un objeto y buscar por su id en especial para actualizar, si si continua su proceso , si no , retorna una excepcion, si todo es correcto y encuentra por el Id, con los setters y la variable que nos ayuda a encontrar la tarea a actualizar junto obteniendo el titulo para asi realizar covalencia y actualizar según lo q obtenga y luego retornar con el metodo save del repository y dentro el nombre de la variable del objeto que llama de punto de refencia la clase model.

Controller

Esta ultima capa de la arquitectura, nos sirve para recibir todos los datos y almacenarlo de forma correcta como parte final de proceso, usamos listas tambien , usamos @RestController para indicar que devuelva JSON no html, y un sub-endpoint para luego sobre otra sub ruta en especial poder crear puntos de refencia en especial para cada uno de nuestros datos creados , usando metodos get , post , put , delete, para interactuar en peticiones CRUD, @RequestMapping para mapear respuestas HTTP,

Igual constructor ahora para instanciar el Service, y get mapping para obtener todas las tareas usando el service, y recordar que esta funcion es de tipo List con el Model , ya que necesita instanciar varios elementos en una lista , con getMapping, luego ese mismo con /{id} de punto de referencia , esta funcion para recibir un estado de peticion (200, 201, 404, 500) , para obtener el id por tarea y la anotacion PathVariable que indicara la variable de la llave primaria a interactiva, si es un estado Ok, usara el metodo de las reglas del negocio para obtener la tarea por id junto a su tipo de dato acompañado con la anotacion. Ahora las otras operaciones Crud para crearla igual pero necesitamos un cuerpo raw para poder inyectarle datos y que los almacene usamos requestBody y el model usamos los metodos del service , para actualizar usaremos ambas anotaciones con los metodos necesarios, y de entrada los dos tipos de datos necesarios y para borrar igualmente aunque ahora es de tipo Void lo que recibe de respuesta la entidad del status , ya que no necesita, y usa el metodo igual y retorna la respuesta de la entidad sin contenido alguno solo la respuesta.

Fin.